Chapter 14. Stream Processing

Kafka was traditionally seen as a powerful message bus, capable of delivering streams of events but without processing or transformation capabilities. Kafka's reliable stream delivery capabilities make it a perfect source of data for stream processing systems. Apache Storm, Apache Spark Streaming, Apache Flink, Apache Samza, and many more stream processing systems were built with Kafka often being their only reliable data source.

With the increased popularity of Apache Kafka, first as a simple message bus and later as a data integration system, many companies had a system containing many streams of interesting data, stored for long amounts of time and perfectly ordered, just waiting for some stream processing framework to show up and process them. In other words, in the same way that data processing was significantly more difficult before databases were invented, stream processing was held back by the lack of a stream processing platform.

Starting from version 0.10.0, Kafka does more than provide a reliable source of data streams to every popular stream processing framework. Now Kafka includes a powerful stream processing library as part of its collection of client libraries, called Kafka Streams (or sometimes Streams API). This allows developers to consume, process, and produce events in their own apps, without relying on an external processing framework.

We'll begin the chapter by explaining what we mean by stream processing (since this term is frequently misunderstood), then discuss some of the basic concepts of stream processing and the design patterns that are common to all stream processing systems. We'll then dive into Apache Kafka's stream processing library—its goals and architecture. We'll give a small example of how to use Kafka Streams to calculate a moving average of stock prices. We'll then discuss other examples of good stream processing use cases and finish off the chapter by providing a few criteria you can use when choosing which stream processing framework (if any) to use with Apache Kafka.

This chapter is intended as just a quick introduction to the large and fascinating world of stream processing and Kafka Streams. There are entire

books written on these subjects.

Some books cover the basic concepts of stream processing from a data architecture perspective:

- *Making Sense of Stream Processing* by Martin Kleppmann (O'Reilly) discusses the benefits of rethinking applications as stream processing applications and how to reorient data architectures around the idea of event streams.
- *Streaming Systems* by Tyler Akidau, Slava Chernyak, and Reuven Lax (O'Reilly) is a great general introduction to the topic of stream processing and some of the basic ideas in the space.
- *Flow Architectures* by James Urquhart (O'Reilly) is targeted at CTOs and discusses the implications of stream processing to the business.

Other books go into specific details of specific frameworks:

- *Mastering Kafka Streams and ksqlDB* by Mitch Seymour (O'Reilly)
- *Kafka Streams in Action* by William P. Bejeck Jr. (Manning)
- *Event Streaming with Kafka Streams and ksqlDB* by William P. Bejeck Jr. (Manning)
- *Stream Processing with Apache Flink* by Fabian Hueske and Vasiliki Kalavri (O'Reilly)
- *Stream Processing with Apache Spark* by Gerard Maas and Francois Garillot (O'Reilly)

Finally, Kafka Streams is still an evolving framework. Every major release deprecates APIs and modifies semantics. This chapter documents APIs and semantics as of Apache Kafka 2.8. We avoided using any API that was planned for deprecation in release 3.0, but our discussion of join semantics and timestamp handling does not include any of the changes planned for release 3.0.

What Is Stream Processing?

There is a lot of confusion about what stream processing means. Many definitions mix up implementation details, performance requirements, data models, and many other aspects of software engineering. A similar thing has happened in the world of relational databases—the abstract definitions of the relational model are getting forever entangled in the implementation details and specific limitations of the popular database engines.

The world of stream processing is still evolving, and just because a specific popular implementation does things in specific ways or has specific limitations doesn't mean that those details are an inherent part of processing streams of data.

Let's start at the beginning: What is a data stream (also called an *event stream* or *streaming data*)? First and foremost, a *data stream* is an abstraction representing an unbounded dataset. *Unbounded* means infinite and ever growing. The dataset is unbounded because over time, new records keep arriving. This definition is used by Google, Amazon, and pretty much everyone else.

Note that this simple model (a stream of events) can be used to represent just about every business activity we care to analyze. We can look at a stream of credit card transactions, stock trades, package deliveries, network events going through a switch, events reported by sensors in manufacturing equipment, emails sent, moves in a game, etc. The list of examples is endless because pretty much everything can be seen as a sequence of events.

There are a few other attributes of the event streams model, in addition to its unbounded nature:

Event streams are ordered

There is an inherent notion of which events occur before or after other events. This is clearest when looking at financial events. A sequence in which you first put money in your account and later spend the money is very different from a sequence at which you first spend the money and later cover your debt by depositing money back. The latter will incur overdraft charges, while the former will not. Note that this is one of the differences between an event stream and a database table—records in a table are always considered unordered, and the "order by" clause of SQL is not part of the relational model; it was added to assist in reporting.

Immutable data records

Events, once occurred, can never be modified. A financial transaction that is canceled does not disappear. Instead, an additional event is written to the stream, recording a cancelation of a previous transaction. When a customer returns merchandise to a shop, we don't delete the fact that the merchandise was sold to them earlier, rather we record the return as an additional event. This is another difference between a data stream and a database table—we can delete or update records in a table, but those are all additional

transactions that occur in the database, and as such can be recorded in a stream of events that records all transactions. If you are familiar with binlogs, WALs, or redo logs in databases, you can see that if we insert a record into a table and later delete it, the table will no longer contain the record, but the redo log will contain two transactions—the insert and the delete.

Event streams are replayable

This is a desirable property. While it is easy to imagine nonreplayable streams (TCP packets streaming through a socket are generally nonreplayable), for most business applications, it is critical to be able to replay a raw stream of events that occurred months (and sometimes years) earlier. This is required to correct errors, try new methods of analysis, or perform audits. This is the reason we believe Kafka made stream processing so successful in modern businesses—it allows capturing and replaying a stream of events. Without this capability, stream processing would not be more than a lab toy for data scientists.

It is worth noting that neither the definition of event streams nor the attributes we later listed say anything about the data contained in the events or the number of events per second. The data differs from system to system—events can be tiny (sometimes only a few bytes) or very large (XML messages with many headers); they can also be completely unstructured key-value pairs, semi-structured JSON, or structured Avro or Protobuf messages. While it is often assumed that data streams are "big data" and involve millions of events per second, the same techniques we'll discuss apply equally well (and often better) to smaller streams of events with only a few events per second or minute.

Now that we know what event streams are, it's time to make sure we understand stream processing. Stream processing refers to the ongoing processing of one or more event streams. Stream processing is a programming paradigm—just like request-response and batch processing. Let's look at how different programming paradigms compare to get a better understanding of how stream processing fits into software architectures:

Request-response

This is the lowest-latency paradigm, with response times ranging from submilliseconds to a few milliseconds, usually with the expectation that response times will be highly consistent. The mode of processing is usually blocking—an app sends a request and waits for the processing system to respond. In the database world, this paradigm is known as *online transaction processing* (OLTP). Point-

of-sale systems, credit card processing, and time-tracking systems typically work in this paradigm.

Batch processing

This is the high-latency/high-throughput option. The processing system wakes up at set times—every day at 2:00 a.m., every hour on the hour, etc. It reads all required input (either all data available since the last execution, all data from the beginning of month, etc.), writes all required output, and goes away until the next time it is scheduled to run. Processing times range from minutes to hours, and users expect to read stale data when they are looking at results. In the database world, these are the data warehouse and business intelligence systems—data is loaded in huge batches once a day, reports are generated, and users look at the same reports until the next data load occurs. This paradigm often has great efficiency and economy of scale, but in recent years, businesses need the data available in shorter timeframes in order to make decision-making more timely and efficient. This puts huge pressure on systems that were written to exploit economy of scale—not to provide low-latency reporting.

Stream processing

This is a continuous and nonblocking option. Stream processing fills the gap between the request-response world, where we wait for events that take two milliseconds to process, and the batch processing world, where data is processed once a day and takes eight hours to complete. Most business processes don't require an immediate response within milliseconds but can't wait for the next day either. Most business processes happen continuously, and as long as the business reports are updated continuously and the line of business apps can continuously respond, the processing can proceed without anyone waiting for a specific response within milliseconds. Business processes such as alerting on suspicious credit transactions or network activity, adjusting prices in real-time based on supply and demand, or tracking deliveries of packages are all a natural fit for continuous but nonblocking processing.

It is important to note that the definition doesn't mandate any specific framework, API, or feature. As long as we are continuously reading data from an unbounded dataset, doing something to it, and emitting output, we are doing stream processing. But the processing has to be continuous and ongoing. A process that starts every day at 2:00 a.m., reads 500 records from the stream, outputs a result, and goes away doesn't quite cut it as far as stream processing goes.

Stream Processing Concepts

Stream processing is very similar to any type of data processing—we write code that receives data, does something with the data (a few transformations, aggregates, enrichments, etc.), and then place the result somewhere. However, there are some key concepts that are unique to stream processing and often cause confusion when someone who has data processing experience first attempts to write stream processing applications. Let's take a look at a few of those concepts.

Topology

A stream processing application includes one or more processing topologies. A processing topology starts with one or more source streams that are passed through a graph of stream processors connected through event streams, until results are written to one or more sink streams. Each stream processor is a computational step applied to the stream of events in order to transform the events. Examples of some stream processors we'll use in our examples are filter, count, group-by, and left-join. We often visualize stream processing applications by drawing the processing nodes and connecting them with arrows to show how events flow from one node to the next as the application is processing data.

Time

Time is probably the most important concept in stream processing and often the most confusing. For an idea of how complex time can get when discussing distributed systems, we recommend Justin Sheehy's excellent paper, "There Is No Now". In the context of stream processing, having a common notion of time is critical because most stream applications perform operations on time windows. For example, our stream application might calculate a moving five-minute average of stock prices. In that case, we need to know what to do when one of our producers goes offline for two hours due to network issues and returns with two hours' worth of data—most of the data will be relevant for five-minute time windows that have long passed and for which the result was already calculated and stored.

Stream processing systems typically refer to the following notions of time:

Event time

This is the time the events we are tracking occurred and the record was created—the time a measurement was taken, an item was sold at a shop, a user viewed a page on our website, etc. In version 0.10.0 and later, Kafka automatically adds the current time to producer records at the time they are created. If this does not match the application's notion of *event time*, such as in cases where the Kafka record is created based on a database record sometime after the event occurred, then we recommend adding the event time as a field in the record itself so that both timestamps will be available for later processing. Event time is usually the time that matters most when processing stream data.

Log append time

This is the time the event arrived at the Kafka broker and was stored there, also called *ingestion time*. In version 0.10.0 and later, Kafka brokers will automatically add this time to records they receive if Kafka is configured to do so or if the records arrive from older producers and contain no timestamps. This notion of time is typically less relevant for stream processing, since we are usually interested in the times the events occurred. For example, if we calculate the number of devices produced per day, we want to count devices that were actually produced on that day, even if there were network issues and the event only arrived to Kafka the following day. However, in cases where the real event time was not recorded, log append time can still be used consistently because it does not change after the record was created, and assuming no delays in the pipeline, it can be a reasonable approximation of event time.

Processing time

This is the time at which a stream processing application received the event in order to perform some calculation. This time can be milliseconds, hours, or days after the event occurred. This notion of time assigns different timestamps to the same event depending on exactly when each stream processing application happened to read the event. It can even differ for two threads in the same application! Therefore, this notion of time is highly unreliable and best avoided.

Kafka Streams assigns time to each event based on the `TimestampExtractor` interface. Developers of Kafka Streams applications can use different implementations of this interface, which can use either of the three time semantics explained previously or a completely different choice of timestamp, including extracting a timestamp from the contents of the event itself.

When Kafka Streams writes output to a Kafka topic, it assigns a timestamp to each event based on the following rules:

- When the output record maps directly to an input record, the output record will use the same timestamp as the input.
- When the output record is a result of an aggregation, the timestamp of the output record will be the maximum timestamp used in the aggregation.
- When the output record is a result of joining two streams, the timestamp of the output record is the largest of the two records being joined. When a stream and a table are joined, the timestamp from the stream record is used.
- Finally, if the output record was generated by a Kafka Streams function that generates data in a specific schedule regardless of input, such as `punctuate()`, the output timestamp will depend on the current internal times of the stream processing app.

When using the Kafka Streams lower-level processing API rather than the DSL, Kafka Streams includes APIs for manipulating the timestamps of records directly, so developers can implement timestamp semantics that match the required business logic of the application.

MIND THE TIME ZONE

When working with time, it is important to be mindful of time zones. The entire data pipeline should standardize on a single time zone; otherwise, results of stream operations will be confusing and often meaningless. If you must handle data streams with different time zones, you need to make sure you can convert events to a single time zone before performing operations on time windows. Often this means storing the time zone in the record itself.

State

As long as we only need to process each event individually, stream processing is a very simple activity. For example, if all we need to do is read a stream of online shopping transactions from Kafka, find the transactions over $10,000, and email the relevant salesperson, we can probably write this in just few lines of code using a Kafka consumer and SMTP library.

Stream processing becomes really interesting when we have operations that involve multiple events: counting the number of events by type, moving averages, joining two streams to create an enriched stream of information, etc. In those cases, it is not enough to look at each event by itself; we need to keep track of more information—how many events of each

type did we see this hour, all events that require joining, sums, averages, etc. We call this information a *state*.

It is often tempting to store the state in variables that are local to the stream processing app, such as a simple hash table to store moving counts. In fact, we did just that in many examples in this book. However, this is not a reliable approach for managing state in stream processing because when the stream processing application is stopped or crashes, the state is lost, which changes the results. This is usually not the desired outcome, so care should be taken to persist the most recent state and recover it when restarting the application.

Stream processing refers to several types of state:

Local or internal state

State that is accessible only by a specific instance of the stream processing application. This state is usually maintained and managed with an embedded, in-memory database running within the application. The advantage of local state is that it is extremely fast. The disadvantage is that we are limited to the amount of memory available. As a result, many of the design patterns in stream processing focus on ways to partition the data into substreams that can be processed using a limited amount of local state.

External state

State that is maintained in an external data store, often a NoSQL system like Cassandra. The advantages of an external state are its virtually unlimited size and the fact that it can be accessed from multiple instances of the application or even from different applications. The downside is the extra latency and complexity introduced with an additional system, as well as availability—the application needs to handle the possibility that the external system is not available. Most stream processing apps try to avoid having to deal with an external store, or at least limit the latency overhead by caching information in the local state and communicating with the external store as rarely as possible. This usually introduces challenges with maintaining consistency between the internal and external state.

Stream-Table Duality

We are all familiar with database tables. A *table* is a collection of records, each identified by its primary key and containing a set of attributes as defined by a schema. Table records are mutable (i.e., tables allow update

and delete operations). Querying a table allows checking the state of the data at a specific point in time. For example, by querying the `CUSTOMERS_CONTACTS` table in a database, we expect to find current contact details for all our customers. Unless the table was specifically designed to include history, we will not find their past contacts in the table.

Unlike tables, streams contain a history of changes. A *stream* is a string of events wherein each event caused a change. A table contains a current state of the world, which is the result of many changes. From this description, it is clear that streams and tables are two sides of the same coin—the world always changes, and sometimes we are interested in the events that caused those changes, whereas other times we are interested in the current state of the world. Systems that allow us to transition back and forth between the two ways of looking at data are more powerful than systems that support just one.

To convert a table to a stream, we need to capture the changes that modify the table. Take all those `insert`, `update`, and `delete` events and store them in a stream. Most databases offer change data capture (CDC) solutions for capturing these changes, and there are many Kafka connectors that can pipe those changes into Kafka where they will be available for stream processing.

To convert a stream to a table, we need to apply all the changes that the stream contains. This is also called *materializing* the stream. We create a table, either in memory, in an internal state store, or in an external database, and start going over all the events in the stream from beginning to end, changing the state as we go. When we finish, we have a table representing a state at a specific time that we can use.

Suppose we have a store selling shoes. A stream representation of our retail activity can be a stream of events:

- "Shipment arrived with red, blue, and green shoes."
- "Blue shoes sold."
- "Red shoes sold."
- "Blue shoes returned."
- "Green shoes sold."

If we want to know what our inventory contains right now or how much money we made until now, we need to materialize the view. Figure 14-1 shows that we currently have 299 red shoes. If we want to know how busy the store is, we can look at the entire stream and see that there were

four customer events today. We may also want to investigate why the blue shoes were returned.

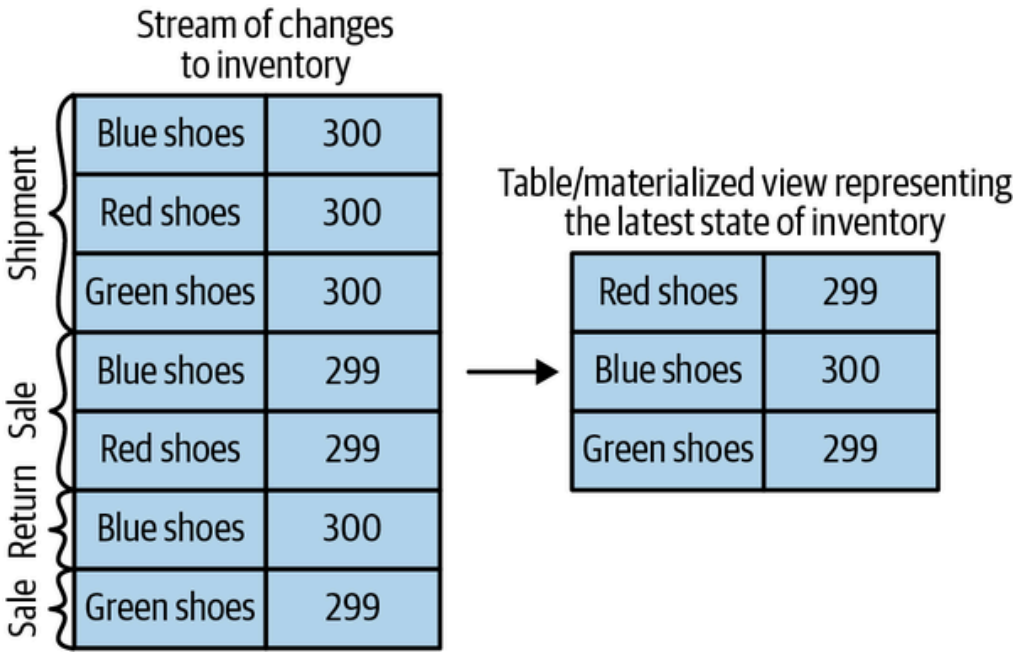

Figure 14-1. Materializing inventory changes

Time Windows

Most operations on streams are windowed operations, operating on slices of time: moving averages, top products sold this week, 99th percentile load on the system, etc. Join operations on two streams are also windowed—we join events that occurred at the same slice of time. Very few people stop and think about the type of window they want for their operations. For example, when calculating moving averages, we want to know:

Size of the window

Do we want to calculate the average of all events in every five-minute window? Every 15-minute window? Or the entire day? Larger windows are smoother but they lag more—if the price increases, it will take longer to notice than with a smaller window. Kafka Streams also includes a *session window*, where the size of the window is defined by a period of inactivity. The developer defines a session gap, and all events that arrive continuously with gaps smaller than the defined session gap belong to the same session. A gap in arrivals will define a new session, and all events arriving after the gap, but before the next gap, will belong to the new session.

How often the window moves (advance interval)

Five-minute averages can update every minute, second, or every time there is a new event. Windows for which the size is a fixed

time interval are called *hopping windows*. When the advance interval is equal to the window size, it is called a *tumbling window*.

How long the window remains updatable (grace period)

Our five-minute moving average calculated the average for the 00:00–00:05 window. Now, an hour later, we are getting a few more input records with their *event time* showing 00:02. Do we update the result for the 00:00–00:05 period? Or do we let bygones be bygones? Ideally, we'll be able to define a certain time period during which events will get added to their respective time slice. For example, if the events were delayed up to four hours, we should recalculate the results and update. If events arrive later than that, we can ignore them.

Windows can be aligned to clock time—i.e., a five-minute window that moves every minute will have the first slice as 00:00–00:05 and the second as 00:01–00:06. Or it can be unaligned and simply start whenever the app started, and then the first slice can be 03:17–03:22. See Figure 14-2 for the difference between these two types of windows.

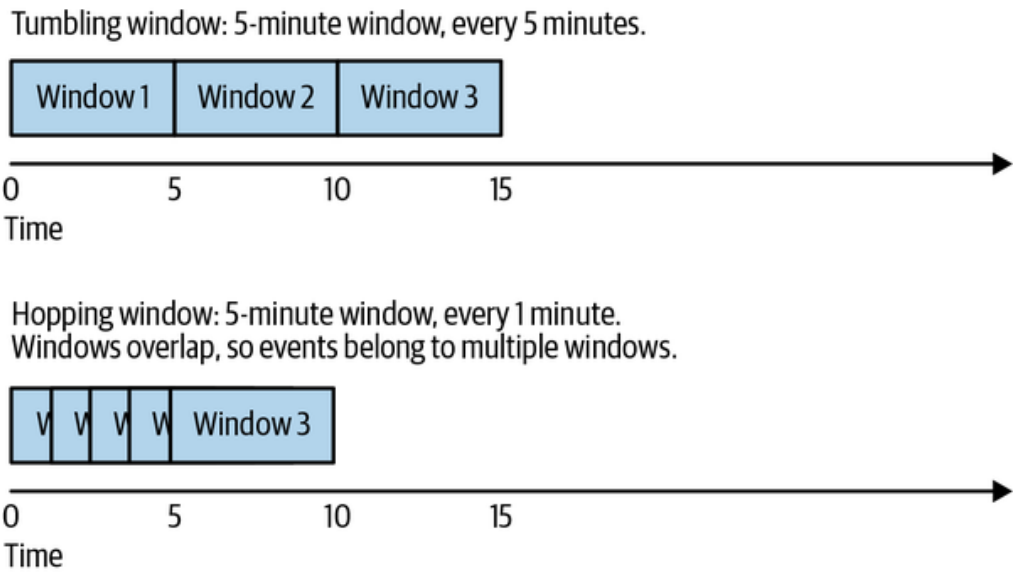

Figure 14-2. Tumbling window versus hopping window

Processing Guarantees

A key requirement for stream processing applications is the ability to process each record exactly once, regardless of failures. Without exactly-once guarantees, stream processing can't be used in cases where accurate results are needed. As discussed in detail in Chapter 8, Apache Kafka has support for exactly-once semantics with a transactional and idempotent producer. Kafka Streams uses Kafka's transactions to implement exactly-once guarantees for stream processing applications. Every application that uses the Kafka Streams library can enable exactly-once guarantees

by setting `processing.guarantee` to `exactly_once`. Kafka Streams version 2.6 or later includes a more efficient exactly-once implementation that requires Kafka brokers of version 2.5 or later. This efficient implementation can be enabled by setting `processing.guarantee` to `exactly_once_beta`.

Stream Processing Design Patterns

Every stream processing system is different—from the basic combination of a consumer, processing logic, and producer, to involved clusters like Spark Streaming with its machine learning libraries, and much in between. But there are some basic design patterns, which are known solutions to common requirements of stream processing architectures. We'll review a few of those well-known patterns and show how they are used with a few examples.

Single-Event Processing

The most basic pattern of stream processing is the processing of each event in isolation. This is also known as a *map/filter pattern* because it is commonly used to filter unnecessary events from the stream or transform each event. (The term *map* is based on the map/reduce pattern in which the map stage transforms events and the reduce stage aggregates them.)

In this pattern, the stream processing app consumes events from the stream, modifies each event, and then produces the events to another stream. An example is an app that reads log messages from a stream and writes `ERROR` events into a high-priority stream and the rest of the events into a low-priority stream. Another example is an application that reads events from a stream and modifies them from JSON to Avro. Such applications do not need to maintain state within the application because each event can be handled independently. This means that recovering from app failures or load-balancing is incredibly easy as there is no need to recover state; we can simply hand off the events to another instance of the app to process.

This pattern can be easily handled with a simple producer and consumer, as seen in Figure 14-3.

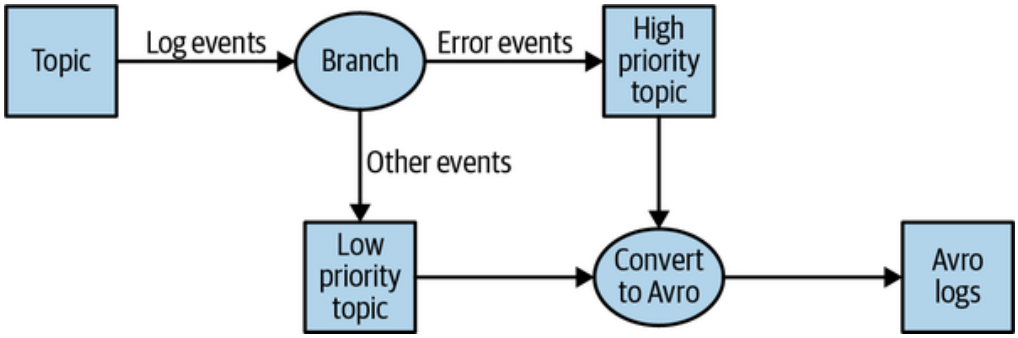

Figure 14-3. Single-event processing topology

Processing with Local State

Most stream processing applications are concerned with aggregating information, especially window aggregation. An example of this is finding the minimum and maximum stock prices for each day of trading and calculating a moving average.

These aggregations require maintaining a *state*. In our example, in order to calculate the minimum and average price each day, we need to store the minimum value, the sum, and the number of records we've seen up until the current time.

All this can be done using *local* state (rather than a shared state) because each operation in our example is a *group by* aggregate. That is, we perform the aggregation per stock symbol, not on the entire stock market in general. We use a Kafka partitioner to make sure that all events with the same stock symbol are written to the same partition. Then, each instance of the application will get all the events from the partitions that are assigned to it (this is a Kafka consumer guarantee). This means that each instance of the application can maintain state for the subset of stock symbols that are written to the partitions that are assigned to it. See Figure 14-4.

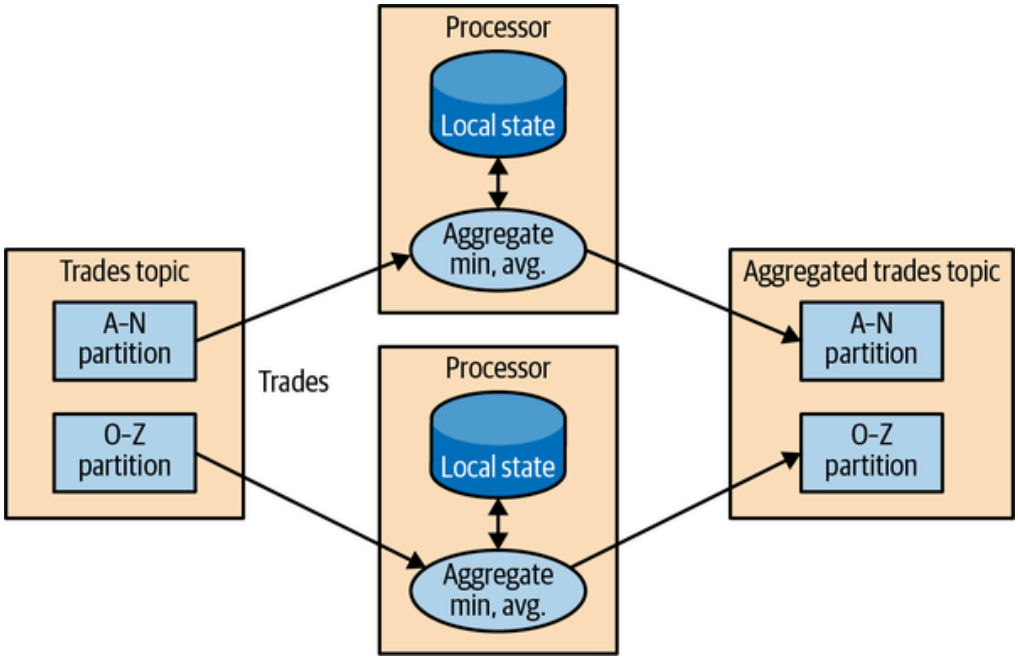

Figure 14-4. Topology for event processing with local state

Stream processing applications become significantly more complicated when the application has local state. There are several issues a stream processing application must address:

Memory usage

The local state ideally fits into the memory available to the application instance. Some local stores allow spilling to disk, but this has significant performance impact.

Persistence

We need to make sure the state is not lost when an application instance shuts down and that the state can be recovered when the instance starts again or is replaced by a different instance. This is something that Kafka Streams handles very well—local state is stored in-memory using embedded RocksDB, which also persists the data to disk for quick recovery after restarts. But all the changes to the local state are also sent to a Kafka topic. If a stream's node goes down, the local state is not lost—it can be easily re-created by rereading the events from the Kafka topic. For example, if the local state contains "current minimum for IBM = 167.19," we store this in Kafka so that later we can repopulate the local cache from this data. Kafka uses log compaction for these topics to make sure they don't grow endlessly and that re-creating the state is always feasible.

Rebalancing

Partitions sometimes get reassigned to a different consumer. When this happens, the instance that loses the partition must store the

last good state, and the instance that receives the partition must know to recover the correct state.

Stream processing frameworks differ in how much they help the developer manage the local state they need. If our application requires maintaining local state, we make sure to check the framework and its guarantees. We'll include a short comparison guide at the end of the chapter, but as we all know, software changes quickly and stream processing frameworks doubly so.

Multiphase Processing/Repartitioning

Local state is great if we need a *group by* type of aggregate. But what if we need a result that uses all available information? For example, suppose we want to publish the top 10 stocks each day—the 10 stocks that gained the most from opening to closing during each day of trading. Obviously, nothing we do locally on each application instance is enough because all the top 10 stocks could be in partitions assigned to other instances. What we need is a two-phase approach. First, we calculate the daily gain/loss for each stock symbol. We can do this on each instance with a local state. Then we write the results to a new topic with a single partition. This partition will be read by a single application instance that can then find the top 10 stocks for the day. The second topic, which contains just the daily summary for each stock symbol, is obviously much smaller with significantly less traffic than the topics that contain the trades themselves, and therefore it can be processed by a single instance of the application. Sometimes more steps are needed to produce the result. See Figure 14-5.

This type of multiphase processing is very familiar to those who write MapReduce code, where you often have to resort to multiple reduce phases. If you've ever written map-reduce code, you'll remember that you needed a separate app for each reduce step. Unlike MapReduce, most stream processing frameworks allow including all steps in a single app, with the framework handling the details of which application instance (or worker) will run each step.

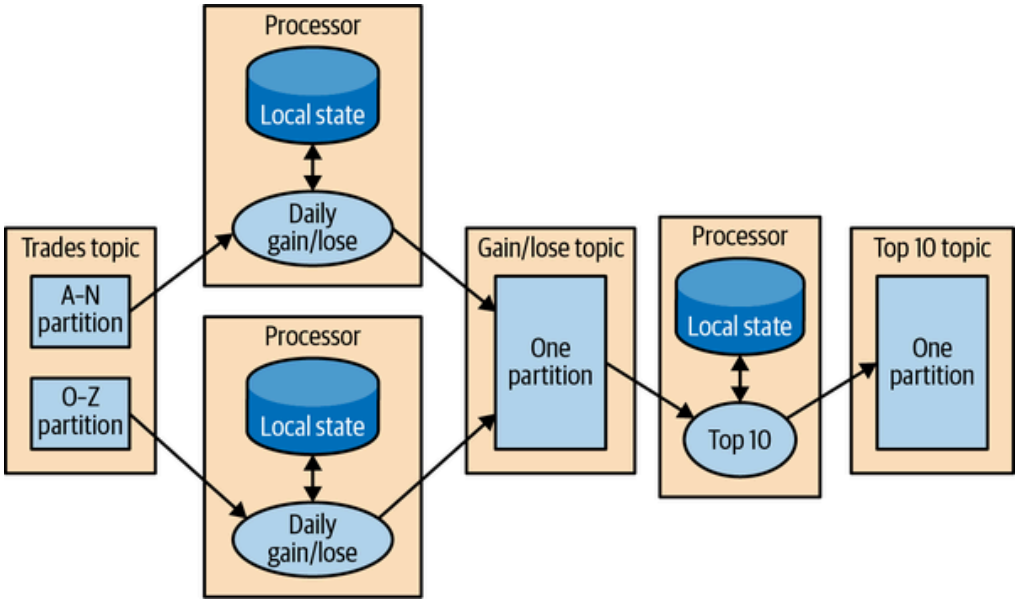

Figure 14-5. Topology that includes both local state and repartitioning steps

Processing with External Lookup: Stream-Table Join

Sometimes stream processing requires integration with data external to the stream—validating transactions against a set of rules stored in a database or enriching clickstream information with data about the users who clicked.

The obvious idea on how to perform an external lookup for data enrichment is something like this: for every click event in the stream, look up the user in the profile database and write an event that includes the original click, plus the user age and gender, to another topic. See Figure 14-6.

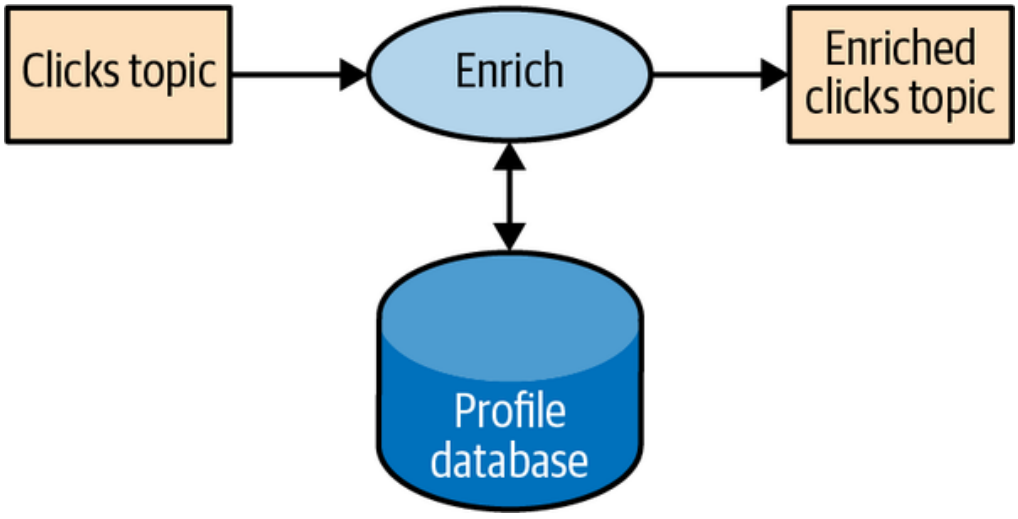

Figure 14-6. Stream processing that includes an external data source

The problem with this obvious idea is that an external lookup adds significant latency to the processing of every record—usually between 5 and 15 milliseconds. In many cases, this is not feasible. Often the additional load this places on the external data store is also not acceptable—stream pro-

cessing systems can often handle 100K–500K events per second, but the database can only handle perhaps 10K events per second at reasonable performance. There is also added complexity around availability—our application will need to handle situations when the external DB is not available.

To get good performance and availability, we need to cache the information from the database in our stream processing application. Managing this cache can be challenging though—how do we prevent the information in the cache from getting stale? If we refresh events too often, we are still hammering the database, and the cache isn't helping much. If we wait too long to get new events, we are doing stream processing with stale information.

But if we can capture all the changes that happen to the database table in a stream of events, we can have our stream processing job listen to this stream and update the cache based on database change events. Capturing changes to the database as events in a stream is known as *change data capture* (CDC), and Kafka Connect has multiple connectors capable of performing CDC and converting database tables to a stream of change events. This allows us to keep our own private copy of the table and be notified whenever there is a database change event so we can update our own copy accordingly. See Figure 14-7.

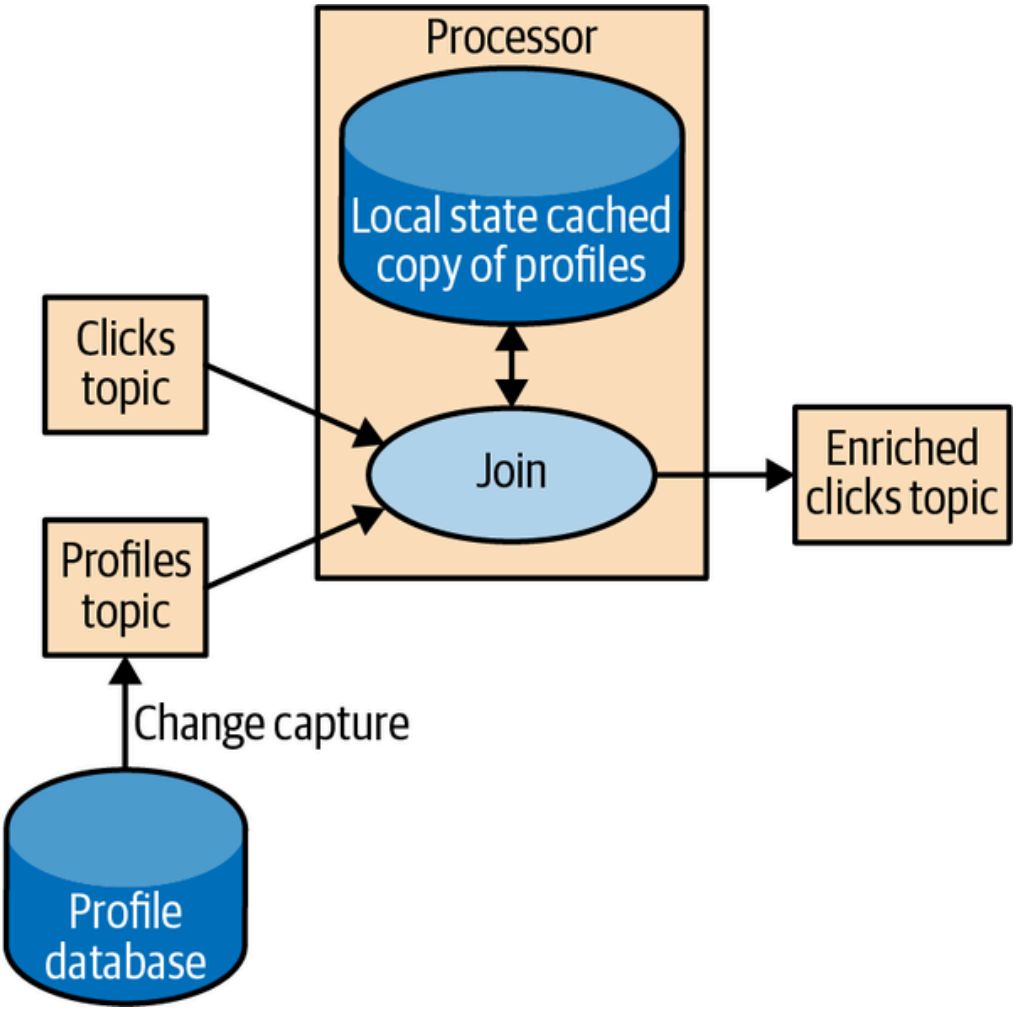

Figure 14-7. Topology joining a table and a stream of events, removing the need to involve an external data source in stream processing

Then, when we get click events, we can look up the `user_id` at our local state and enrich the event. And because we are using a local state, this scales a lot better and will not affect the database and other apps using it.

We refer to this as a *stream-table join* because one of the streams represents changes to a locally cached table.

Table-Table Join

In the previous section we discussed how a table and a stream of update events are equivalent. We've discussed in detail how this works when joining a stream and a table. There is no reason why we can't have those materialized tables in both sides of the join operation.

Joining two tables is always nonwindowed and joins the current state of both tables at the time the operation is performed. With Kafka Streams, we can perform an `equi-join`, in which both tables have the same key that is partitioned in the same way, and therefore the join operation can be efficiently distributed between a large number of application instances and machines.

Kafka Streams also supports `foreign-key join` of two tables—the key of one stream or table is joined with an arbitrary field from another stream or table. You can learn more about how it works in "Crossing the Streams", a talk from Kafka Summit 2020, or the more in-depth blog post.

Streaming Join

Sometimes we want to join two real event streams rather than a stream with a table. What makes a stream "real"? If you recall the discussion at the beginning of the chapter, streams are unbounded. When we use a stream to represent a table, we can ignore most of the history in the stream because we only care about the current state in the table. But when we join two streams, we are joining the entire history, trying to match events in one stream with events in the other stream that have the same key and happened in the same time windows. This is why a streaming join is also called a *windowed join*.

For example, let's say that we have one stream with search queries that people entered into our website and another stream with clicks, which include clicks on search results. We want to match search queries with the results they clicked on so that we will know which result is most popular for which query. Obviously, we want to match results based on the search term but only match them within a certain time window. We assume the result is clicked seconds after the query was entered into our search engine. So we keep a small, few-seconds-long window on each stream and match the results from each window. See Figure 14-8.

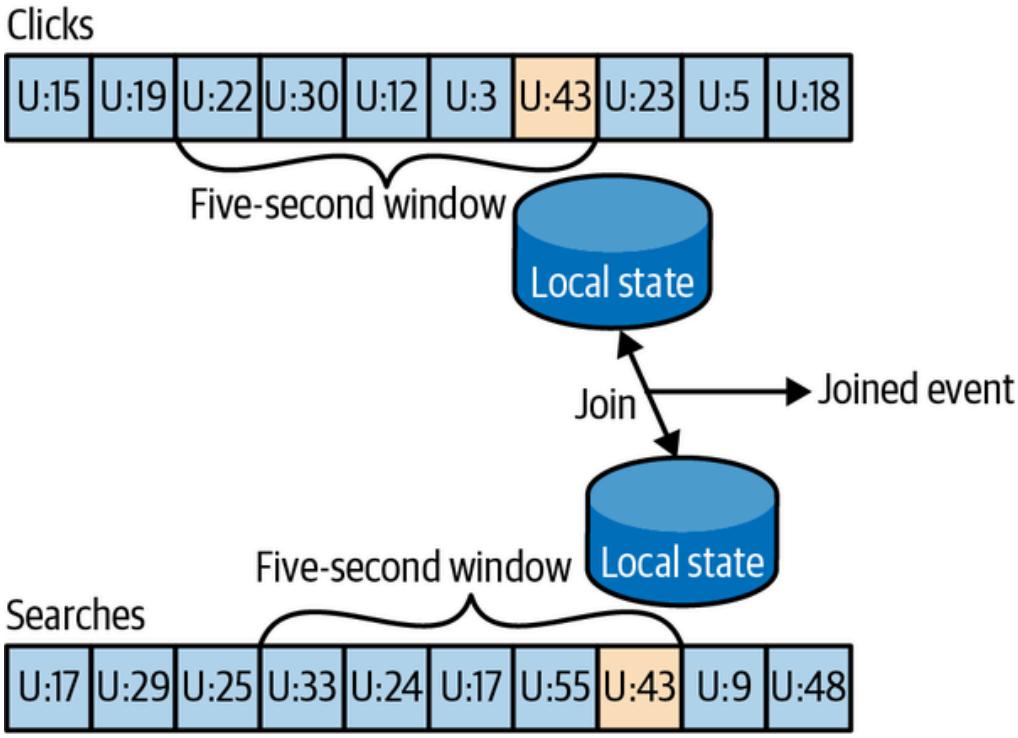

Figure 14-8. Joining two streams of events; these joins always involve a moving time window

Kafka Streams supports `equi-joins`, where streams, queries, and clicks are partitioned on the same keys, which are also the join keys. This way, all the click events from `user_id:42` end up in partition 5 of the clicks topic, and all the search events for `user_id:42` end up in partition 5 of the search topic. Kafka Streams then makes sure that partition 5 of both topics is assigned to the same task. So this task sees all the relevant events for `user_id:42`. It maintains the join window for both topics in its embedded RocksDB state store, and this is how it can perform the join.

Out-of-Sequence Events

Handling events that arrive at the stream at the wrong time is a challenge not just in stream processing but also in traditional ETL systems. Out-of-sequence events happen quite frequently and expectedly in IoT scenarios (Figure 14-9). For example, a mobile device loses WiFi signal for a few hours and sends a few hours' worth of events when it reconnects. This also happens when monitoring network equipment (a faulty switch doesn't send diagnostics signals until it is repaired) or manufacturing (network connectivity in plants is notoriously unreliable, especially in developing countries).

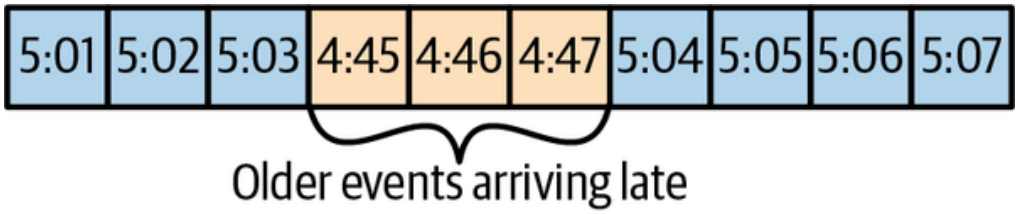

Figure 14-9. Out-of-sequence events

Our streams applications need to be able to handle those scenarios. This typically means the application has to do the following:

- Recognize that an event is out of sequence—this requires that the application examines the event time and discovers that it is older than the current time.
- Define a time period during which it will attempt to reconcile out-of-sequence events. Perhaps a three-hour delay should be reconciled, and events over three weeks old can be thrown away.
- Have an in-band capability to reconcile this event. This is the main difference between streaming apps and batch jobs. If we have a daily batch job and a few events arrived after the job completed, we can usually just rerun yesterday's job and update the events. With stream processing, there is no "rerun yesterday's job"—the same continuous process needs to handle both old and new events at any given moment.

- Be able to update results. If the results of the stream processing are written into a database, a *put* or *update* is enough to update the results. If the stream app sends results by email, updates may be trickier.

Several stream processing frameworks, including Google's Dataflow and Kafka Streams, have built-in support for the notion of event time independent of the processing time, and the ability to handle events with event times that are older or newer than the current processing time. This is typically done by maintaining multiple aggregation windows available for update in the local state and giving developers the ability to configure how long to keep those window aggregates available for updates. Of course, the longer the aggregation windows are kept available for updates, the more memory is required to maintain the local state.

The Kafka Streams API always writes aggregation results to result topics. Those are usually `compacted topics`, which means that only the latest value for each key is preserved. In case the results of an aggregation window need to be updated as a result of a late event, Kafka Streams will simply write a new result for this aggregation window, which will effectively replace the previous result.

Reprocessing

The last important pattern is reprocessing events. There are two variants of this pattern:

- We have an improved version of our stream processing application. We want to run the new version of the application on the same event stream as the old, produce a new stream of results that does not replace the first version, compare the results between the two versions, and at some point move clients to use the new results instead of the existing ones.
- The existing stream processing app is buggy. We fix the bug, and we want to reprocess the event stream and recalculate our results

The first use case is made simple by the fact that Apache Kafka stores the event streams in their entirety for long periods of time in a scalable data store. This means that having two versions of a stream processing application writing two result streams only requires the following:

- Spinning up the new version of the application as a new consumer group
- Configuring the new version to start processing from the first offset of the input topics (so it will get its own copy of all events in the input

streams)
- Letting the new application continue processing, and switching the client applications to the new result stream when the new version of the processing job has caught up

The second use case is more challenging—it requires "resetting" an existing app to start processing back at the beginning of the input streams, resetting the local state (so we won't mix results from the two versions of the app), and possibly cleaning the previous output stream. While Kafka Streams has a tool for resetting the state for a stream processing app, our recommendation is to try to use the first method whenever sufficient capacity exists to run two copies of the app and generate two result streams. The first method is much safer—it allows switching back and forth between multiple versions and comparing results between versions, and doesn't risk losing critical data or introducing errors during the cleanup process.

Interactive Queries

As discussed previously, stream processing applications have state, and this state can be distributed among many instances of the application. Most of the time the users of stream processing applications get the results of the processing by reading them from an output topic. In some cases, however, it is desirable to take a shortcut and read the results from the state store itself. This is common when the result is a table (e.g., the top 10 best-selling books) and the stream of results is really a stream of updates to this table—it is much faster and easier to just read the table directly from the stream processing application state.

Kafka Streams includes flexible APIs for querying the state of a stream processing application.

Kafka Streams by Example

To demonstrate how these patterns are implemented in practice, we'll show a few examples using the Apache Kafka Streams API. We are using this specific API because it is relatively simple to use and it ships with Apache Kafka, which we already have access to. It is important to remember that the patterns can be implemented in any stream processing framework and library—the patterns are universal, but the examples are specific.

Apache Kafka has two stream APIs—a low-level Processor API and a high-level Streams DSL. We will use the Kafka Streams DSL in our examples. The DSL allows us to define the stream processing application by defining a chain of transformations to events in the streams. Transformations can be as simple as a filter or as complex as a stream-to-stream join. The lower-level API allows us to create our own transformations. To learn more about the low-level Processor API, the developer guide has detailed information, and the presentation "Beyond the DSL" is a great introduction.

An application that uses the DSL API always starts with using the `StreamsBuilder` to create a processing *topology*—a directed acyclic graph (DAG) of transformations that are applied to the events in the streams. Then we create a `KafkaStreams` execution object from the topology. Starting the `KafkaStreams` object will start multiple threads, each applying the processing topology to events in the stream. The processing will conclude when we close the `KafkaStreams` object.

We'll look at a few examples that use Kafka Streams to implement some of the design patterns we just discussed. A simple word count example will be used to demonstrate the map/filter pattern and simple aggregates. Then we'll move to an example where we calculate different statistics on stock market trades, which will allow us to demonstrate window aggregations. Finally, we'll use ClickStream enrichment as an example to demonstrate streaming joins.

Word Count

Let's walk through an abbreviated word count example for Kafka Streams. You can find the full example on GitHub.

The first thing you do when creating a stream processing app is configure Kafka Streams. Kafka Streams has a large number of possible configurations, which we won't discuss here, but you can find them in the documentation. In addition, you can configure the producer and consumer embedded in Kafka Streams by adding any producer or consumer config to the `Properties` object:

```
public class WordCountExample {

    public static void main(String[] args) throws Exception{

        Properties props = new Properties();
        props.put(StreamsConfig.APPLICATION_ID_CONFIG,
          "wordcount"); ❶
        props.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG,
```

```
        "localhost:9092"); ❷
      props.put(StreamsConfig.DEFAULT_KEY_SERDE_CLASS_CONFIG,
        Serdes.String().getClass().getName()); ❸
      props.put(StreamsConfig.DEFAULT_VALUE_SERDE_CLASS_CONFIG,
        Serdes.String().getClass().getName());
```

❶ Every Kafka Streams application must have an application ID. It is used to coordinate the instances of the application and also when naming the internal local stores and the topics related to them. This name must be unique for each Kafka Streams application working with the same Kafka cluster.

❷ The Kafka Streams application always reads data from Kafka topics and writes its output to Kafka topics. As we'll discuss later, Kafka Streams applications also use Kafka for coordination. So we had better tell our app where to find Kafka.

❸ When reading and writing data, our app will need to serialize and deserialize, so we provide default Serde classes. If needed, we can override these defaults later when building the streams topology.

Now that we have the configuration, let's build our streams topology:

```
StreamsBuilder builder = new StreamsBuilder(); ❶

KStream<String, String> source =
  builder.stream("wordcount-input");


final Pattern pattern = Pattern.compile("\\W+");

KStream<String, String> counts  = source.flatMapValues(value->
  Arrays.asList(pattern.split(value.toLowerCase()))) ❷
        .map((key, value) -> new KeyValue<String,
           String>(value, value))
        .filter((key, value) -> (!value.equals("the"))) ❸
        .groupByKey() ❹
        .count().mapValues(value->
           Long.toString(value)).toStream();❺
counts.to("wordcount-output"); ❻
```

❶ We create a `StreamsBuilder` object and start defining a stream by pointing at the topic we'll use as our input.

❷ Each event we read from the source topic is a line of words; we split it up using a regular expression into a series of individual

words. Then we take each word (currently a value of the event record) and put it in the event record key so it can be used in a group-by operation.

❸ We filter out the word *the*, just to show how easy filtering is.

❹ And we group by key, so we now have a collection of events for each unique word.

❺ We count how many events we have in each collection. The result of counting is a `Long` data type. We convert it to a `String` so it will be easier for humans to read the results.

❻ Only one thing left—write the results back to Kafka.

Now that we have defined the flow of transformations that our application will run, we just need to…run it:

```
KafkaStreams streams = new KafkaStreams(builder.build(), props); ❶

streams.start(); ❷

// usually the stream application would be running forever,
// in this example we just let it run for some time and stop
Thread.sleep(5000L);

streams.close(); ❸
```

❶ Define a `KafkaStreams` object based on our topology and the properties we defined.

❷ Start Kafka Streams.

❸ After a while, stop it.

That's it! In just a few short lines, we demonstrated how easy it is to implement a single event processing pattern (we applied a map and a filter on the events). We repartitioned the data by adding a group-by operator and then maintained simple local state when we counted the number of records that have each word as a key. Then we maintained simple local state when we counted the number of times each word appeared.

At this point, we recommend running the full example. The README in the GitHub repository contains instructions on how to run the example.

Note that we can run the entire example on our machine without installing anything except Apache Kafka. If our input topic contains multiple partitions, we can run multiple instances of the `WordCount` application (just run the app in several different terminal tabs), and we have our first Kafka Streams processing cluster. The instances of the `WordCount` application talk to one another and coordinate the work. One of the biggest barriers to entry for some stream processing frameworks is that local mode is very easy to use, but then to run a production cluster, we need to install YARN or Mesos, then install the processing framework on all those machines, and then learn how to submit our app to the cluster. With the Kafka's Streams API, we just start multiple instances of our app—and we have a cluster. The exact same app is running on our development machine and in production.

Stock Market Statistics

The next example is more involved—we will read a stream of stock market trading events that include the stock ticker, ask price, and ask size. In stock market trades, *ask price* is what a seller is asking for, whereas *bid price* is what the buyer is suggesting to pay. *Ask size* is the number of shares the seller is willing to sell at that price. For simplicity of the example, we'll ignore bids completely. We also won't include a timestamp in our data; instead, we'll rely on event time populated by our Kafka producer.

We will then create output streams that contain a few windowed statistics:

- Best (i.e., minimum) ask price for every five-second window
- Number of trades for every five-second window
- Average ask price for every five-second window

All statistics will be updated every second.

For simplicity, we'll assume our exchange only has 10 stock tickers trading in it. The setup and configuration are very similar to those we used in "Word Count":

```
Properties props = new Properties();
props.put(StreamsConfig.APPLICATION_ID_CONFIG, "stockstat");
props.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG, Constants.BROKER);
props.put(StreamsConfig.DEFAULT_KEY_SERDE_CLASS_CONFIG,
  Serdes.String().getClass().getName());
props.put(StreamsConfig.DEFAULT_VALUE_SERDE_CLASS_CONFIG,
  TradeSerde.class.getName());
```

The main difference is the Serde classes used. In "Word Count", we used strings for both key and value, and therefore used the `Serdes.String()` class as a serializer and deserializer for both. In this example, the key is still a string, but the value is a `Trade` object that contains the ticker symbol, ask price, and ask size. In order to serialize and deserialize this object (and a few other objects we used in this small app), we used the Gson library from Google to generate a JSON serializer and deserializer from our `Java` object. Then we created a small wrapper that created a Serde object from those. Here is how we created the Serde:

```
static public final class TradeSerde extends WrapperSerde<Trade> {
    public TradeSerde() {
        super(new JsonSerializer<Trade>(),
          new JsonDeserializer<Trade>(Trade.class));
    }
}
```

Nothing fancy, but remember to provide a Serde object for every object you want to store in Kafka—input, output, and, in some cases, intermediate results. To make this easier, we recommend generating these Serdes through a library like Gson, Avro, Protobuf, or something similar.

Now that we have everything configured, it's time to build our topology:

```
KStream<Windowed<String>, TradeStats> stats = source
    .groupByKey() ❶
    .windowedBy(TimeWindows.of(Duration.ofMillis(windowSize))
                           .advanceBy(Duration.ofSeconds(1))) ❷
    .aggregate( ❸
        () -> new TradeStats(),
        (k, v, tradestats) -> tradestats.add(v), ❹
        Materialized.<String, TradeStats, WindowStore<Bytes, byte[]>>
            as("trade-aggregates") ❺
           .withValueSerde(new TradeStatsSerde())) ❻
    .toStream() ❼
    .mapValues((trade) -> trade.computeAvgPrice()); ❽

stats.to("stockstats-output",
    Produced.keySerde(
      WindowedSerdes.timeWindowedSerdeFrom(String.class, windowSize)));
```

❶ We start by reading events from the input topic and performing a `groupByKey()` operation. Despite its name, this operation does not do any grouping. Rather, it ensures that the stream of events is partitioned based on the record key. Since we wrote the data into a

topic with a key and didn't modify the key before calling `groupByKey()`, the data is still partitioned by its key—so this method does nothing in this case.

❷ We define the window—in this case, a window of five seconds, advancing every second.

❸ After we ensure correct partitioning and windowing, we start the aggregation. The `aggregate` method will split the stream into overlapping windows (a five-second window every second) and then apply an aggregate method on all the events in the window. The first parameter this method takes is a new object that will contain the results of the aggregation— `Tradestats`, in our case. This is an object we created to contain all the statistics we are interested in for each time window—minimum price, average price, and number of trades.

❹ We then supply a method for actually aggregating the records—in this case, an `add` method of the `Tradestats` object is used to update the minimum price, number of trades, and total prices in the window with the new record.

❺ As mentioned in "Stream Processing Design Patterns", windowing aggregation requires maintaining a state and a local store in which the state will be maintained. The last parameter of the aggregate method is the configuration of the state store. `Materialized` is the store configuration object, and we configure the store name as `trade-aggregates`. This can be any unique name.

❻ As part of the state store configuration, we also provide a Serde object for serializing and deserializing the results of the aggregation (the `Tradestats` object).

❼ The results of the aggregation is a *table* with the ticker and the time window as the primary key and the aggregation result as the value. We are turning the table back into a stream of events.

❽ The last step is to update the average price—right now the aggregation results include the sum of prices and number of trades. We go over these records and use the existing statistics to calculate average price so we can include it in the output stream.

❾ And finally, we write the results back to the `stockstats-output` stream. Since the results are part of a windowing operation, we create a `WindowedSerde` that stores the result in a windowed data

format that includes the window timestamp. The window size is passed as part of the Serde, even though it isn't used in the serialization (deserialization requires the window size, because only the start time of the window is stored in the output topic).

After we define the flow, we use it to generate a `KafkaStreams` object and run it, just like we did in "Word Count".

This example shows how to perform windowed aggregation on a stream—probably the most popular use case of stream processing. One thing to notice is how little work was needed to maintain the local state of the aggregation—just provide a Serde and name the state store. Yet this application will scale to multiple instances and automatically recover from a failure of each instance by shifting processing of some partitions to one of the surviving instances. We will see more on how it is done in "Kafka Streams: Architecture Overview".

As usual, you can find the complete example, including instructions for running it, on GitHub.

ClickStream Enrichment

The last example will demonstrate streaming joins by enriching a stream of clicks on a website. We will generate a stream of simulated clicks, a stream of updates to a fictional profile database table, and a stream of web searches. We will then join all three streams to get a 360-degree view into each user activity. What did the users search for? What did they click as a result? Did they change their "interests" in their user profile? These kinds of joins provide a rich data collection for analytics. Product recommendations are often based on this kind of information—the user searched for bikes, clicked on links for "Trek," and is interested in travel, so we can advertise bikes from Trek, helmets, and bike tours to exotic locations like Nebraska.

Since configuring the app is similar to the previous examples, let's skip this part and take a look at the topology for joining multiple streams:

```
KStream<Integer, PageView> views =
    builder.stream(Constants.PAGE_VIEW_TOPIC,
      Consumed.with(Serdes.Integer(), new PageViewSerde())); ❶
KStream<Integer, Search> searches =
    builder.stream(Constants.SEARCH_TOPIC,
      Consumed.with(Serdes.Integer(), new SearchSerde()));
KTable<Integer, UserProfile> profiles =
    builder.table(Constants.USER_PROFILE_TOPIC,
```

```
        Consumed.with(Serdes.Integer(), new ProfileSerde())); ❷


KStream<Integer, UserActivity> viewsWithProfile = views.leftJoin(profiles
                (page, profile) -> {
                    if (profile != null)
                        return new UserActivity(
                          profile.getUserID(), profile.getUserName(),
                          profile.getZipcode(), profile.getInterests(),
                          "", page.getPage()); ❹
                    else
                       return new UserActivity(
                         -1, "", "", null, "", page.getPage());
                    });

KStream<Integer, UserActivity> userActivityKStream =
    viewsWithProfile.leftJoin(searches, ❺
      (userActivity, search) -> {
          if (search != null)
              userActivity.updateSearch(search.getSearchTerms()); ❻
          else
              userActivity.updateSearch("");
          return userActivity;
      },
      JoinWindows.of(Duration.ofSeconds(1)).before(Duration.ofSeconds(0))
                  StreamJoined.with(Serdes.Integer(),  ❽
                                    new UserActivitySerde(),
                                    new SearchSerde()));
```

❶ First, we create a streams objects for the two streams we want to join—clicks and searches. When we create the stream object, we pass the input topic and the key and value Serde that will be used when consuming records out of the topic and deserializing them into input objects.

❷ We also define a `KTable` for the user profiles. A `KTable` is a materialized store that is updated through a stream of changes.

❸ Then we enrich the stream of clicks with user profile information by joining the stream of events with the profile table. In a stream-table join, each event in the stream receives information from the cached copy of the profile table. We are doing a left-join, so clicks without a known user will be preserved.

❹ This is the `join` method—it takes two values, one from the stream and one from the record, and returns a third value. Unlike in databases, we get to decide how to combine the two values into one re-

sult. In this case, we created one `activity` object that contains both the user details and the page viewed.

❺ Next, we want to `join` the click information with searches performed by the same user. This is still a left `join`, but now we are joining two streams, not streaming to a table.

❻ This is the `join` method—we simply add the search terms to all the matching page views.

❼ This is the interesting part—a *stream-to-stream join* is a join with a time window. Joining all clicks and searches for each user doesn't make much sense—we want to join each search with clicks that are related to it, that is, clicks that occurred a short period of time after the search. So we define a join window of one second. We invoke `of` to create a window of one second before and after each search, and then we call `before` with a zero-seconds interval to make sure we only join clicks that happen one second after each search and not before. The results will include relevant clicks, search terms, and the user profile. This will allow a full analysis of searches and their results.

❽ We define the Serde of the join result here. This includes a Serde for the key that both sides of the join have in common and the Serde for both values that will be included in the result of the join. In this case, the key is the user ID, so we use a simple `Integer` Serde.

After we define the flow, we use it to generate a `KafkaStreams` object and run it, just like we did in "Word Count".

This example shows two different join patterns possible in stream processing. One joins a stream with a table to enrich all streaming events with information in the table. This is similar to joining a fact table with a dimension when running queries on a data warehouse. The second example joins two streams based on a time window. This operation is unique to stream processing.

As usual, you can find the complete example, including instructions for running it, on GitHub.

Kafka Streams: Architecture Overview

The examples in the previous section demonstrated how to use the Kafka Streams API to implement a few well-known stream processing design patterns. But to understand better how Kafka's Streams library actually works and scales, we need to peek under the covers and understand some of the design principles behind the API.

Building a Topology

Every streams application implements and executes one *topology*. Topology (also called DAG, or directed acyclic graph, in other stream processing frameworks) is a set of operations and transitions that every event moves through from input to output. Figure 14-10 shows the topology in "Word Count".

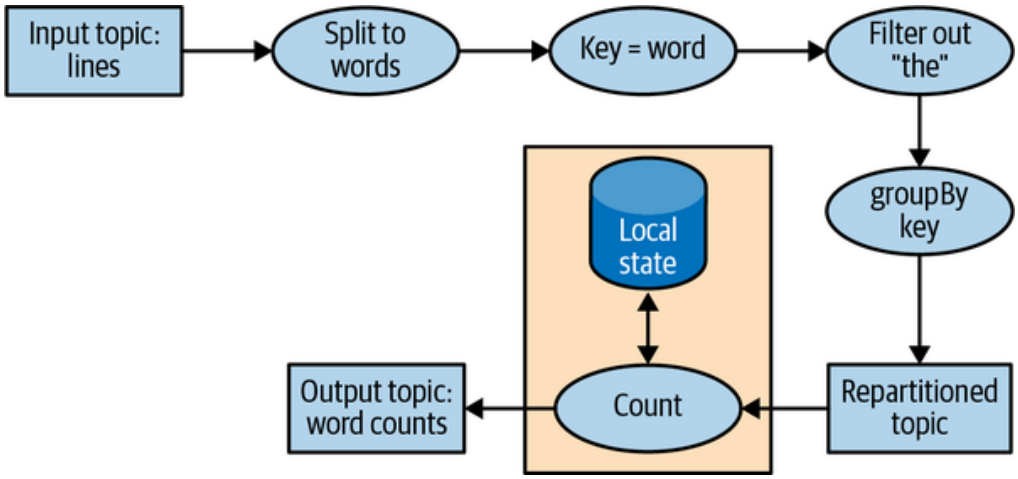

Figure 14-10. Topology for the word-count stream processing example

Even a simple app has a nontrivial topology. The topology is made up of processors—those are the nodes in the topology graph (represented by circles in our diagram). Most processors implement an operation of the data—filter, map, aggregate, etc. There are also source processors, which consume data from a topic and pass it on, and sink processors, which take data from earlier processors and produce it to a topic. A topology always starts with one or more source processors and finishes with one or more sink processors.

Optimizing a Topology

By default, Kafka Streams executes applications that were built with the DSL API by mapping each DSL method independently to a lower-level equivalent. By evaluating each DSL method independently, opportunities to optimize the overall resulting topology were missed.

However, note that the execution of a Kafka Streams application is a three-step process:

1. The logical topology is defined by creating `KStream` and `KTable` objects and performing DSL operations, such as filter and join, on them.
2. `StreamsBuilder.build()` generates a physical topology from the logical topology.
3. `KafkaStreams.start()` executes the topology—this is where data is consumed, processed, and produced.

The second step, where the physical topology is generated from the logical definitions, is where overall optimizations to the plan can be applied.

Currently, Apache Kafka only contains a few optimizations, mostly around reusing topics where possible. These can be enabled by setting `StreamsConfig.TOPOLOGY_OPTIMIZATION` to `StreamsConfig.OPTIMIZE` and calling `build(props)`. If you only call `build()` without passing the config, optimization is still disabled. It is recommended to test applications with and without optimizations and to compare execution times and volumes of data written to Kafka, and of course, validate that the results are identical in various known scenarios.

Testing a Topology

Generally speaking, we want to test software before using it in scenarios where its successful execution is important. Automated testing is considered the gold standard. Repeatable tests that are evaluated every time a change is made to a software application or library enable fast iterations and easier troubleshooting.

We want to apply the same kind of methodology to our Kafka Streams applications. In addition to automated end-to-end tests that run the stream processing application against a staging environment with generated data, we'll want to also include faster, lighter-weight, and easier-to-debug unit and integration tests.

The main testing tool for Kafka Streams applications is `TopologyTestDriver`. Since its introduction in version 1.1.0, its API has undergone significant improvements, and versions since 2.4 are convenient and easy to use. These tests look like normal unit tests. We define input data, produce it to mock input topics, run the topology with the test driver, read the results from mock output topics, and validate the result by comparing it to expected values.

We recommend using the `TopologyTestDriver` for testing stream processing applications, but since it does not simulate Kafka Streams caching behavior (an optimization not discussed in this book, entirely unrelated to the state store itself, which is simulated by this framework), there are entire classes of errors that it will not detect.

Unit tests are typically complemented by integration tests, and for Kafka Streams, there are two popular integration test frameworks: `EmbeddedKafkaCluster` and `Testcontainers`. The former runs Kafka brokers inside the JVM that runs the tests, while the latter runs Docker containers with Kafka brokers (and many other components, as needed for the tests). `Testcontainers` is recommended, since by using Docker it fully isolates Kafka, its dependencies, and its resource usage from the application we are trying to test.

This is just a short overview of Kafka Streams testing methodologies. We recommend reading the "Testing Kafka Streams—A Deep Dive" blog post for deeper explanations and detailed code examples of topologies and tests.

Scaling a Topology

Kafka Streams scales by allowing multiple threads of executions within one instance of the application and by supporting load balancing between distributed instances of the application. We can run the Streams application on one machine with multiple threads or on multiple machines; in either case, all active threads in the application will balance the work involved in data processing.

The Streams engine parallelizes execution of a topology by splitting it into tasks. The number of tasks is determined by the Streams engine and depends on the number of partitions in the topics that the application processes. Each task is responsible for a subset of the partitions: the task will subscribe to those partitions and consume events from them. For every event it consumes, the task will execute all the processing steps that apply to this partition in order before eventually writing the result to the sink. Those tasks are the basic unit of parallelism in Kafka Streams, because each task can execute independently of others. See Figure 14-11.

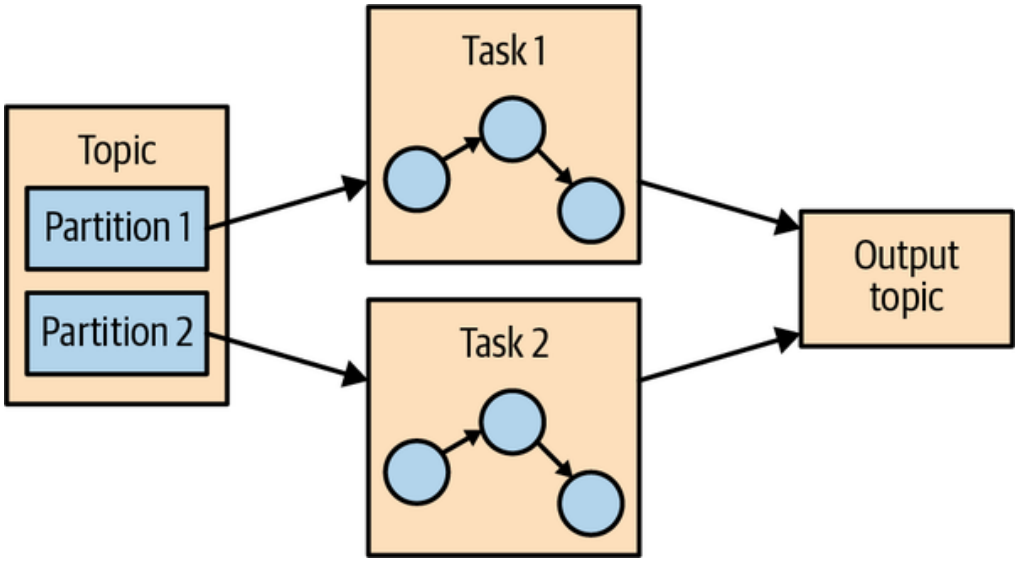

Figure 14-11. Two tasks running the same topology—one for each partition in the input topic

The developer of the application can choose the number of threads each application instance will execute. If multiple threads are available, every thread will execute a subset of the tasks that the application creates. If multiple instances of the application are running on multiple servers, different tasks will execute for each thread on each server. This is the way streaming applications scale: we will have as many tasks as we have partitions in the topics we are processing. If we want to process faster, add more threads. If we run out of resources on the server, start another instance of the application on another server. Kafka will automatically coordinate work—it will assign each task its own subset of partitions, and each task will independently process events from those partitions and maintain its own local state with relevant aggregates if the topology requires this. See Figure 14-12.

Sometimes a processing step may require input from multiple partitions, which could create dependencies between tasks. For example, if we join two streams, as we did in the ClickStream example in "ClickStream Enrichment", we need data from a partition in each stream before we can emit a result. Kafka Streams handles this situation by assigning all the partitions needed for one join to the same task so that the task can consume from all the relevant partitions and perform the join independently. This is why Kafka Streams currently requires that all topics that participate in a join operation have the same number of partitions and be partitioned based on the join key.

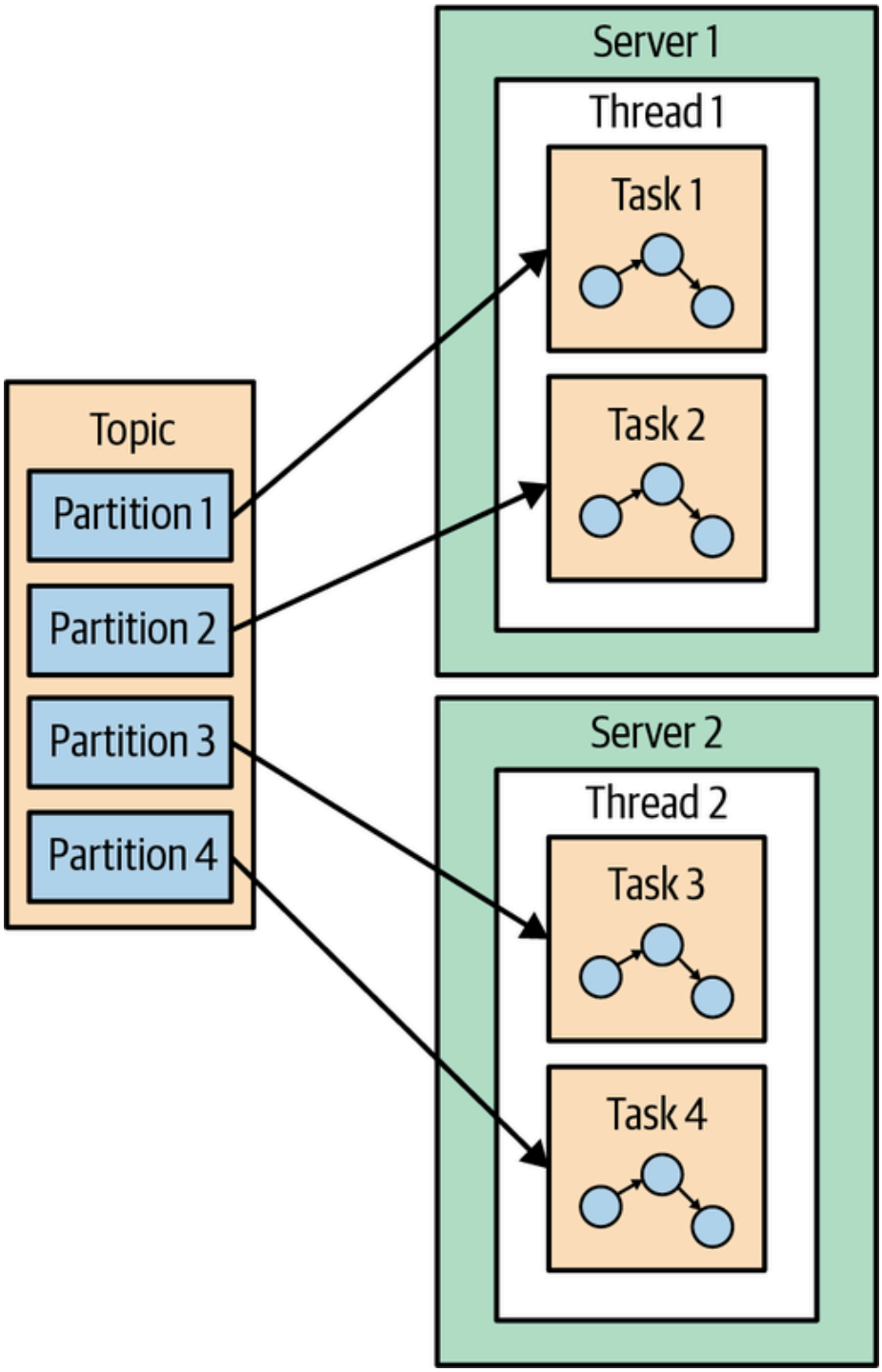

Figure 14-12. The stream processing tasks can run on multiple threads and multiple servers

Another example of dependencies between tasks is when our application requires repartitioning. For instance, in the ClickStream example, all our events are keyed by the user ID. But what if we want to generate statistics per page? Or per zip code? Kafka Streams will repartition the data by zip code and run an aggregation of the data with the new partitions. If task 1 processes the data from partition 1 and reaches a processor that repartitions the data (`groupBy` operation), it will need to *shuffle*, or send events to other tasks. Unlike other stream processor frameworks, Kafka Streams repartitions by writing the events to a new topic with new keys and partitions. Then another set of tasks reads events from the new topic and continues processing. The repartitioning steps break our topology into two subtopologies, each with its own tasks. The second set of tasks depends on the first, because it processes the results of the first subtopology.

However, the first and second sets of tasks can still run independently and in parallel because the first set of tasks writes data into a topic at its own rate and the second set consumes from the topic and processes the events on its own. There is no communication and no shared resources between the tasks, and they don't need to run on the same threads or servers. This is one of the more useful things Kafka does—reduce dependencies between different parts of a pipeline. See Figure 14-13.

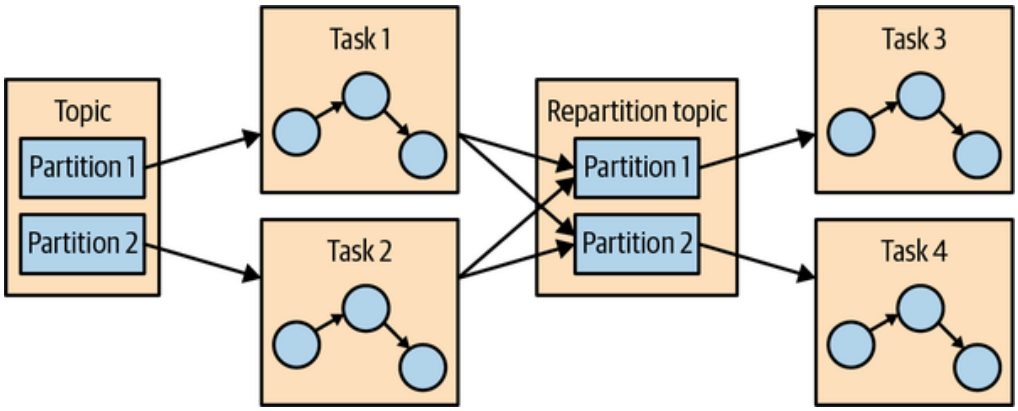

Figure 14-13. Two sets of tasks processing events with a topic for repartitioning events between them

Surviving Failures

The same model that allows us to scale our application also allows us to gracefully handle failures. First, Kafka is highly available, and therefore the data we persist to Kafka is also highly available. So if the application fails and needs to restart, it can look up its last position in the stream from Kafka and continue its processing from the last offset it committed before failing. Note that if the local state store is lost (e.g., because we needed to replace the server it was stored on), the streams application can always re-create it from the change log it stores in Kafka.

Kafka Streams also leverages Kafka's consumer coordination to provide high availability for tasks. If a task failed but there are threads or other instances of the streams application that are active, the task will restart on one of the available threads. This is similar to how consumer groups handle the failure of one of the consumers in the group by assigning partitions to one of the remaining consumers. Kafka Streams benefited from improvements in Kafka's consumer group coordination protocol, such as static group membership and cooperative rebalancing (described in Chapter 4), as well as improvements to Kafka's exactly-once semantics (described in Chapter 8).

While the high-availability methods described here work well in theory, reality introduces some complexity. One important concern is the speed of recovery. When a thread has to start processing a task that used to run

on a failed thread, it first needs to recover its saved state—the current aggregation windows, for instance. Often this is done by rereading internal topics from Kafka in order to warm up Kafka Streams state stores. During the time it takes to recover the state of a failed task, the stream processing job will not make progress on that subset of its data, leading to reduced availability and stale data.

Therefore, reducing recovery time often boils down to reducing the time it takes to recover the state. A key technique is to make sure all Kafka Streams topics are configured for aggressive compaction—by setting a low `min.compaction.lag.ms` and configuring the segment size to 100 MB instead of the default 1 GB (recall that the last segment in each partition, the active segment, is not compacted).

For an even faster recovery, we recommend configuring `standby replica` —those are tasks that simply shadow active tasks in a stream processing application and keep the current state warm on a different server. When failover occurs, they already have the most current state and are ready to continue processing with almost no downtime.

More information on both scalability and high availability in Kafka Streams is available in a a blog post and a Kafka summit talk on the topic.

Stream Processing Use Cases

Throughout this chapter we've learned how to do stream processing—from general concepts and patterns to specific examples in Kafka Streams. At this point it may be worth looking at the common stream processing use cases. As explained in the beginning of the chapter, stream processing—or continuous processing—is useful in cases where we want our events to be processed in quick order rather than wait for hours until the next batch, but also where we are not expecting a response to arrive in milliseconds. This is all true but also very abstract. Let's look at a few real scenarios that can be solved with stream processing:

Customer service

> Suppose that we just reserved a room at a large hotel chain, and we expect an email confirmation and receipt. A few minutes after reserving, when the confirmation still hasn't arrived, we call customer service to confirm our reservation. Suppose the customer service desk tells us, "I don't see the order in our system, but the batch job that loads the data from the reservation system to the hotels and the customer service desk only runs once a day, so please

call back tomorrow. You should see the email within 2–3 business days." This doesn't sound like very good service, yet we've had this conversation more than once with a large hotel chain. What we really want is for every system in the hotel chain to get an update about a new reservation seconds or minutes after the reservation is made, including the customer service center, the hotel, the system that sends email confirmations, the website, etc. We also want the customer service center to be able to immediately pull up all the details about any of our past visits to any of the hotels in the chain, and the reception desk at the hotel to know that we are a loyal customer so they can give us an upgrade. Building all those systems using stream processing applications allows them to receive and process updates in near real time, which makes for a better customer experience. With such a system, the customer would receive a confirmation email within minutes, their credit card would be charged on time, the receipt would be sent, and the service desk could immediately answer their questions regarding the reservation.

Internet of Things

IoT can mean many things—from a home device for adjusting temperature and ordering refills of laundry detergent, to real-time quality control of pharmaceutical manufacturing. A very common use case when applying stream processing to sensors and devices is to try to predict when preventive maintenance is needed. This is similar to application monitoring but applied to hardware and is common in many industries, including manufacturing, telecommunications (identifying faulty cellphone towers), cable TV (identifying faulty box-top devices before users complain), and many more. Every case has its own pattern, but the goal is similar: process events arriving from devices at a large scale and identify patterns that signal that a device requires maintenance. These patterns can be dropped packets for a switch, more force required to tighten screws in manufacturing, or users restarting the box more frequently for cable TV.

Fraud detection

Also known as *anomaly detection*, this is a very wide field that focuses on catching "cheaters" or bad actors in the system. Examples of fraud-detection applications include detecting credit card fraud, stock trading fraud, video-game cheaters, and cybersecurity risks. In all these fields, there are large benefits to catching fraud as early as possible, so a near real-time system that is capable of responding to events quickly—perhaps stopping a bad transaction before it is

even approved—is much preferred to a batch job that detects fraud three days after the fact, when cleanup is much more complicated. This is, again, a problem of identifying patterns in a large-scale stream of events.

In cybersecurity, there is a method known as *beaconing*. When the hacker plants malware inside the organization, it will occasionally reach outside to receive commands. It can be difficult to detect this activity since it can happen at any time and any frequency. Typically, networks are well defended against external attacks but more vulnerable to someone inside the organization reaching out. By processing the large stream of network connection events and recognizing a pattern of communication as abnormal (for example, detecting that this host typically doesn't access those specific IPs), the security organization can be alerted early, before more harm is done.

How to Choose a Stream Processing Framework

When choosing a stream processing framework, it is important to consider the type of application you are planning on writing. Different types of applications call for different stream processing solutions:

Ingest

Where the goal is to get data from one system to another, with some modification to the data to conform to the target system.

Low milliseconds actions

Any application that requires almost immediate response. Some fraud-detection use cases fall within this bucket.

Asynchronous microservices

These microservices perform a simple action on behalf of a larger business process, such as updating the inventory of a store. These applications may need to maintain local state caching events as a way to improve performance.

Near real-time data analytics

These streaming applications perform complex aggregations and joins in order to slice and dice the data and generate interesting, business-relevant insights.

The stream processing system you will choose will depend a lot on the problem you are solving:

- If you are trying to solve an ingest problem, you should reconsider whether you want a stream processing system or a simpler ingest-focused system like Kafka Connect. If you are sure you want a stream processing system, you need to make sure it has both a good selection of connectors and high-quality connectors for the systems you are targeting.
- If you are trying to solve a problem that requires low milliseconds actions, you should also reconsider your choice of streams. Request-response patterns are often better suited to this task. If you are sure you want a stream processing system, then you need to opt for one that supports an event-by-event low-latency model rather than one that focuses on microbatches.
- If you are building asynchronous microservices, you need a stream processing system that integrates well with your message bus of choice (Kafka, hopefully), has change capture capabilities that easily deliver upstream changes to the microservice local state, and has the good support of a local store that can serve as a cache or materialized view of the microservice data.
- If you are building a complex analytics engine, you also need a stream processing system with great support for a local store—this time, not for maintenance of local caches and materialized views but rather to support advanced aggregations, windows, and joins that are otherwise difficult to implement. The APIs should include support for custom aggregations, window operations, and multiple join types.

In addition to use case–specific considerations, there are a few global considerations you should take into account:

Operability of the system

Is it easy to deploy to production? Is it easy to monitor and troubleshoot? Is it easy to scale up and down when needed? Does it integrate well with your existing infrastructure? What if there is a mistake and you need to reprocess data?

Usability of APIs and ease of debugging

I've seen orders-of-magnitude differences in the time it takes to write a high-quality application among different versions of the same framework. Development time and time-to-market are important, so you need to choose a system that makes you efficient.

Makes hard things easy

Almost every system will claim they can do advanced windowed aggregations and maintain local stores, but the question is: do they make it easy for you? Do they handle gritty details around scale and recovery, or do they supply leaky abstractions and make you handle most of the mess? The more a system exposes clean APIs and abstractions and handles the gritty details on its own, the more productive developers will be.

Community

Most stream processing applications you consider are going to be open source, and there's no replacement for a vibrant and active community. Good community means you get new and exciting features on a regular basis, the quality is relatively good (no one wants to work on bad software), bugs get fixed quickly, and user questions get answers in a timely manner. It also means that if you get a strange error and Google it, you will find information about it because other people are using this system and seeing the same issues.

Summary

We started the chapter by explaining stream processing. We gave a formal definition and discussed the common attributes of the stream processing paradigm. We also compared it to other programming paradigms.

We then discussed important stream processing concepts. Those concepts were demonstrated with three example applications written with Kafka Streams.

After going over all the details of these example applications, we gave an overview of the Kafka Streams architecture and explained how it works under the covers. We conclude the chapter, and the book, with several examples of stream processing use cases and advice on how to compare different stream processing frameworks.